

EMS 算法从零理解

1. 先想一个简单问题

你有一个 $300 \times 200 \times 150$ 的箱子，往里塞长方体工件。怎么塞才不浪费空间？

最朴素的想法：每次选一个工件，放到当前最「合适」的角落。

EMS 就是这个思路，但它把「还有哪些空位」用一个空闲空间列表精确记下来。

2. 核心数据结构：Space（空闲空间）

```
Space(x=0, y=0, z=0, dx=300, dy=200, dz=150)
```

就是一个矩形区域：从 (x, y, z) 开始，向 x/y/z 各延伸 (dx, dy, dz) 。

初始状态：整个箱子就是一个 Space。

3. 算法核心循环（EMS 的灵魂）

每一步重复三件事：

第一步：找到「最贴合」的工件-空间组合

遍历所有待放工件 $\times$ 所有空闲空间，计算贴合分数：

$$score = (space.dx - dx) + (space.dy - dy) + (space.dz - dz)$$

score 越小 = 工件放进去后留下的缝隙越小 = 越贴合。这就是 Best-Fit 策略。

找到 score 最小的那个，放进去，工件紧贴空间原点 $(space.x, space.y, space.z)$ 。

第二步：分裂被占用的空间

工件放进空间后，那个空间被占了一部分，剩下的部分要切成子空间。

想象一个方块被从角落挖掉一块，剩下的部分可以切成最多 6 块：



代码对应 `_split_space()` 函数，生成这 6 个方向上的子空间（只保留尺寸 > 0 的）。

第三步：切除相交 + 合并

- **切除相交**：新工件可能穿透到其他空闲空间里，遍历所有空间，如果相交就切除相交部分（`remove_intersections`）
- **合并**（`try_merge_spaces`）：相邻空间如果共享一整面且另两轴对齐，合成一个大的，防止碎片化

三步走完 → 空间列表又回到「互不重叠 + 覆盖所有空域」的状态 → 下一轮继续。

4. 一个具体例子

$300 \times 200 \times 150$ 的箱子，先放一个 $80 \times 60 \times 50$ （J05 的一种姿态）：

初始：1 个 `space = (0,0,0, 300,200,150)`

放入 J05 在 $(0,0,0)$ ：

→ 分裂原始 `space`，得到：

- 上方： $(0,0,50, 300,200,100)$
- 前方： $(0,60,0, 300,140,50)$
- 右方： $(80,0,0, 220,200,50)$

第二轮：在上方空间 $(0,0,50, 300,200,100)$ 里再放一个...

→ 继续分裂...

5. 多策略

同一批工件，排列顺序不同，结果天差地别。尝试 3 种排序策略：

策略	逻辑
<code>volume_desc</code>	大件先放，小件填空
<code>footprint_desc</code>	底面积大的先放
<code>longest_desc</code>	最长的先放

每个箱子三种都试，选利用率最高的。

6. 一句话总结

EMS = 维护一个互不重叠的空闲空间列表 + 贪心 Best-Fit 选择 + 分裂/切除/合并三步走。

7. 代码对应

打开 `ems.py`，跟着这个顺序看：

序号	内容	<code>ems.py</code> 行号
1	<code>Space</code> 类 — 空闲空间数据结构	第 9 行
2	<code>EMSBin.pack()</code> — 主循环（三步走）	第 118 行
3	<code>_split_space()</code> — 空间切成最多 6 块	第 41 行
4	<code>remove_intersections()</code> — 切除相交	第 82 行
5	<code>try_merge_spaces()</code> — 合并碎片	第 90 行